Code-Layout, Readability and Reusability

Date	26 June 2026
Team ID	N/A
Project Name	Voice Based Concept Understanding Analyser
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used Throughout t the code	Yes	Consistent indentation is maintained throughout the Streamlit and Python code.
2	Proper File Structure	Files and folders are logically organized	Yes	Project files are organized into modules such as app.py, speech_processing.py, concept_analyzer.py, audio_features.py, and report_generator.py.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Variables like audio_file, transcript, similarity_score, and understanding_level clearly describe their purpose.
4	Function / Method Names	Functions are descriptively named	Yes	Functions such as transcribe_audio(), analyze_concept(), extract_audio_features(), and generate_report() are meaningfully named.
5	Code Comments	Inline and block comments explain logic	Partial	Important logic is commented for easier understanding and maintenance.
6	Modular Design	Code is split into reusable functions/modules	Yes	Speech processing, semantic analysis, scoring, and report generation are implemented as reusable modules.

7	No Redundant Code	Duplicate or unused code is removed	Yes	Duplicate code is avoided by reusing helper functions across the application.
8	Error Handling	Exceptions and errors are handled carefully	Yes	Exceptions are handled for invalid audio files, transcription failures, and unsupported formats.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	Speech Recognition Module	Python, Whisper	Converts speech to text	High
2	Concept Analysis Module	Python, Sentence Transformers	Semantic similarity evaluation	High
3	Audio Feature Extraction	Python, Librosa	Speech quality and filler word analysis	High
4	Scoring Engine	Python	Calculates understanding score and level	High
5	Report Generator	Python, ReportLab	Generates downloadable PDF reports	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Well-organized modular architecture with separate components.
Readability	5	Clear function names and descriptive variable naming improve readability.
Reusability	5	Core modules can be reused for future voice-based assessment projects. Documentation / Comments
Documentation / Comments	4	Sufficient comments are provided; additional API documentation can be added.
Overall Score	4.8 / 5	Clean, maintainable, modular, and scalable implementation suitable for the Voice Based Concept Understanding Analyser.